Nombre (s) :	Matrícula (s):
Darío Alberto Sánchez Perzabal	A01738266
Santiago Patricio Gómez Ochoa	A01735171
Ximena Ortiz Gómez	A01735100
María José Croche Álvarez	A01734561

Nombre del curso:	Nombre del profesor (es):
Implementación de Robótica Inteligente	Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez Cesar Torres Huitzil
Módulo:	**Actividad:** Reto Semanal 3
Fecha: 20 de abril del 2026	

Resumen

En este mini reto se implementó un sistema de navegación punto a punto en lazo cerrado para el robot Puzzlebot utilizando el framework ROS2. El desarrollo se basó en la integración de tres elementos principales los cuales son un nodo de odometría para la estimación de la pose del robot a partir de los encoders, un controlador proporcional para minimizar el error de la posición y orientación, y un sistema o nodo de generación de trayectorias.

Inicialmente, se implementó el cálculo de la localización del Puzzlebot mediante el modelo cinemático diferencial y el método numérico Euler, lo que permitió obtener en tiempo real la posición y orientación del robot. Posteriormente, se implementó y diseñó un controlador proporcional que a partir del error entre la posición actual y la posición objetivo, generó la velocidad lineal y angular para guiar al robot hacia su objetivo.

En la primera parte del reto se logró que el robot siguiera una trayectoria cuadrada de 2 metros por lado utilizando una secuencia fija de puntos. Asimismo, en la segunda parte el usuario pudo realizar diferentes rutas más complejas que permitieron que el seguimiento de rutas fuera más complejo . Finalmente, se incorporaron estrategias de robustez como saturación de velocidades, tolerancias de error y normalización del ángulo para lograr un comportamiento estable ante el ruido y limitaciones del sistema. Es por ello que los

resultados fueron los esperados y demostraron el correcto funcionamiento del sistema de control en lazo cerrado.

Objetivos

El objetivo del reto semanal 3 es implementar un control de lazo cerrado, es decir con retroalimentación, en este caso, la retroalimentación se da gracias a los encoders de los motores, específicamente con las velocidades angulares. Para ello es necesario desarrollar un nodo en ROS2 en el cual debe implementarse un controlador P, PI o PID. Este nodo *controller* es el encargado de realizar la conversión de *quaternion* a *yaw,* normalizar el ángulo, y sobre todo de calcular los errores tanto de distancia como del ángulo, permitiendo que el robot corrija su posición. En adición a esto, es necesario implementar también un nodo *path_generator*, en el que se generan y publican las posiciones objetivo (goals), creando la trayectoria que debe seguir el robot. Este nodo cuenta con 4 modos diferentes, el primero siendo una trayectoria definida por el usuario constando de 4 coordenadas diferentes. Los modos 2, 3, y 4 son figuras predefinidas, siendo un triángulo, un rombo y un pentágono respectivamente.

Introducción

Uno de los principales retos de la robótica actualmente es lograr que un robot sea capaz de desplazarse de manera autónoma hacia una posición deseada con precisión adaptándose a las condiciones del entorno y a las limitaciones físicas del robot. Para ello, este reto es fundamental para integrar conceptos como la localización, el control de lazo cerrado y la navegación punto a punto, los cuales permiten que en este caso el Puzzlebot tome decisiones en tiempo real a partir de la información recibida y la información que percibe. En el caso del Puzzlebot se aplican normalmente técnicas de navegación punto a punto donde el objetivo es llevar al robot desde una posición inicial hasta una meta en específico en el espacio.

Para ello, resulta importante definir que es la odometría la cual es el estudio de la estimación de la posición de robots o vehículos con ruedas durante la navegación. Para realizar esta estimación se usa información sobre la rotación de las ruedas para estimar cambios en la posición a lo largo del tiempo [1]. Como se comentó anteriormente, la odometría es esencial para la localización en tiempo real, no obstante, su precisión disminuye con el tiempo debido a la acumulación de errores, por lo que es importante aplicar un sistema de control.

De igual manera el cálculo del error es un elemento clave en la elaboración de sistemas de control en lazo cerrado y este es definido como la diferencia entre el estado deseado y el estado actual del robot. Por ende para un robot móvil en un plano 2D este error puede definirse como:

- Error en posición en x: $e_x = x_g - x_r$
- Error en posición en y :$e_y = y_g - y_r$
- Error en orientación: $e_\theta = atan(e_y, e_x) - \theta_r$

Estos términos representan el error en posición y orientación, y son utilizados por el controlador para generar las señales de control necesarias. A partir de estos errores, es posible definir también un error de distancia, el cual simplifica el diseño del controlador. [2]

Por otra parte, un sistema de control en lazo cerrado utiliza retroalimentación para ajustar continuamente la acción del robot. A diferencia del reto anterior donde se aplicó un sistema de control en lazo abierto, este tipo de control permite corregir las desviaciones en tiempo real mejorando precisión y estabilidad en el movimiento del Puzzlebot. Por ende, los controladores de lazo cerrado son P, PI o PID y son ampliamente utilizados debido a su capacidad de minimizar el error entre la posición actual y la deseada.

Es por ello que en el desarrollo de este reto se usó un controlador proporcional (P), el cual es la forma más simple de un control en lazo cerrado dado que la acción de control es directamente proporcional al error actual multiplicado por una ganancia Kp. Este controlador actúa instantáneamente puesto que responde al error actual produciendo una salida proporcional a la magnitud del error. No obstante entre sus principales desventajas es que le cuesta eliminar el error de estado estacionario por sí solo y un aumento excesivo de la ganancia Kp puede hacer el sistema inestable u oscilatorio [3].

Finalmente, un aspecto clave del diseño del controlador es su robustez, la cual se refiere a la capacidad del sistema a mantener un desempeño adecuado ante perturbaciones del entorno, ruido en los sensores y no linealidades del sistema. En sistemas reales como el Puzzlebot factores como el ruido de los encoders, el deslizamiento de la pista y variaciones en la superficie de prueba afectan el comportamiento del robot, por lo que resulta importante ajustar de manera correcta los parámetros del controlador y considerar estos efectos en el diseño.

Solución del problema

Para resolver este reto se diseñó e implementó una arquitectura modular en ROS2 basada en control en lazo cerrado para el Puzzlebot. La solución se desarrolló tomando como base la

metodología propuesta por Manchester Robotics para control de movimiento de un robot diferencial en el cual la posición deseada se compara con la posición estimada del robot con el fin de obtener un error y a partir de este generar señales de control necesarias para guiar el Puzzlebot hasta la meta objetivo. Por ende, en la parte 1 de este reto se pidió que el robot siguiera una trayectoria cuadrada con lados de 2m a partir de una pose inicial (0,0,0), mientras que en la parte 2 de este mini reto se solicitó realizar un nodo generador de trayectoria capaz de publicar múltiples metas mediante un mensaje personalizado.

Por ende, podemos decir que la metodología general de este proyecto se dividió en 3 partes principales. La primera parte consistió en estimar la pose del robot a partir de los encoders de las ruedas mediante odometría. La segunda parte de la metodología consistió en implementar un controlador de posición en lazo cerrado que cuando la poste estimada pudiera calcular el error respecto a la meta y convertirlo en comandos de velocidad linear y angular mediante /cmd _vel. Finalmente la tercera etapa fue implementar un nodo generador de trayectorias para la segunda parte del reto con el fin de publicar las metas o figuras predeterminadas y permitir que el puzzlebot pudiera realizar el seguimiento automático de distintas rutas y que el usuario cambiara parámetros de dichas rutas.

Parte 1

Hablando de la primera parte de reto, el primer componente implementado en la solución de este mini reto fue el nodo *puzzlebot_odometry* el cual es el encargado de estimar la posición y orientación del robot. Este nodo es indispensable para este reto ya que los sensores del Puzzlebot no proporcionan directamente la posición en el plano, sino las velocidades angulares de las ruedas, por lo que es necesario reconstruir la pose mediante un proceso de localización por odometría o dead reckoning. Por ende, hablando específicamente de este nodo, en el código se definieron las constantes físicas del robot las cuales son el radio de la rueda con un valor de 0.05m, una separación entre ruedas de 0.19m y un tiempo de muestreo de 0.02s. Consecuentemente, a partir de las velocidades angulares de las ruedas recibidas en los tópicos */VelocityEnvR* y */VelocityEncL* se calculó la velocidad lineal y la velocidad angular del robot con el modelo cinemático diferencial. Particularmente se utilizaron las siguientes ecuaciones:

$$v = \frac{r(w_r + w_l)}{2} \quad (1)$$

$$\omega = \frac{r(w_r - w_l)}{L} \quad (2)$$

Consecuentemente, una vez obtenidas estas velocidades se actualizó la pose del robot por medio del método numérico de integración de Euler y de esta manera las coordenadas x,y y el ángulo θ fueran recalculados en cada instante de muestreo. Esto permitió estimar la posición del robot a partir de sus velocidades angulares de cada llanta. Lo anterior podemos observar en las siguientes líneas de código implementado en el nodo odometría:

```
v = self.r * (self.wr + self.wl) / 2.0
w = self.r * (self.wr - self.wl) / self.L

self.x += v * np.cos(self.theta) * self.Ts
self.y += v * np.sin(self.theta) * self.Ts
self.theta += w * self.Ts
self.theta = self.normalize_angle(self.theta)
```

De igual manera, además de actualizar la pose, el nodo publica un mensaje de tipo Odometry en el tópico /odom, el cual contiene tanto la posición como la orientación del Puzzlebot. Además en este nodo se realiza la normalización del ángulo utilizando una función basada en atan2 con el objetivo de mantener la orientación en el intervalo $[-\pi, \pi]$ y evitar errores en el cálculo de la orientación del robot.

Consecuentemente, con la localización ya implementada se procedió al desarrollo del controlador de la parte 1 del reto. Para ello se creó el nodo *controller_sqr* cuyo objetivo fue desplazar al robot a través de una secuencia de 4 puntos que describen un cuadrado con medidas de 2m en cada lado. Estos puntos o también conocidos como metas fueron definidos en el archivo de parámetros llamado *puzzlebot_params.yaml*, lo que permitió modificar la trayectoria sin necesidad de alterar directamente el código y mejorar la flexibilidad de la implementación.

El nodo *controller_sqr* se subscribe al tópico /odom para obtener en tiempo real la pose estimada del robot y publica en /cmd_vel los comandos de la velocidad lineal y angular. Por ende, en cada ciclo de control primeramente se calcula el error de la posición respecto a la meta actual en términos de sus componentes cartesianos. A partir de estas diferencias se obtiene el error de distancia calculado entre la posición actual y la meta, y finalmente el error angular se calculó como la diferencia entre el ángulo deseado hacia el objetivo y la orientación actual del robot. En el código estos cálculos se implementaron de la siguiente manera:

```
dx = goal_x - self.x
dy = goal_y - self.y
distance_error = math.sqrt(dx ** 2 + dy ** 2)
desired_theta = math.atan2(dy, dx)
theta_error = self.normalize_angle(desired_theta - self.theta)
```

Consecuentemente, en el caso de esta solución del mini reto, para el control se optó por una estrategia proporcional utilizando una ganancia Kp para el error de distancia y otra Kp para el error angular. En la implementación realizada las ganancias utilizadas para la parte 1 de este reto fueron *kp_dist=0.8* y *kp_theta=2.0,* mientras que las velocidades máximas se limitaron a *v_max = 0.3* y *w_max = 0.7*, datos que fueron calculados en el reto de la semana 1 observando las limitaciones físicas del Puzzlebot. Asimismo, el controlador fue diseñado para que cuando el error angular era mayor que una tolerancia establecida el robot girara sobre su propio eje sin avanzar linealmente, una vez que la orientación se encontraba suficientemente alineada con la meta, se habilitaba el movimiento hacia delante junto con una corrección angular. Esto permitió reducir trayectorias que fueran realizadas de mala manera por el robot y mejorar la estabilidad del seguimiento de los vértices del cuadrado. La parte principal del algoritmo controlador para la parte 1 puede resumirse en el siguiente segmento de código:

```
if abs(theta_error) > self.theta_tolerance:
    cmd.linear.x = 0.0
    cmd.angular.z = self.saturate(self.kp_theta * theta_error, self.w_max)
else:
    cmd.linear.x = self.saturate(self.kp_dist * distance_error, self.v_max)
    cmd.angular.z = self.saturate(self.kp_theta * theta_error, self.w_max)
```

Aunado a lo anterior, una vez que el error de distancias se encontraba por debajo de la tolerancia de llegada, el nodo considera que la meta ha sido alcanzada, por lo que se detiene el robot y avanza hacia el siguiente punto de la trayectoria. Esto se realizó debido a que en la odometría hay limitaciones y también en el espacio hay ruido, por lo que se definió una tolerancia de 0.05m para la trayectoria cuadrada. De igual manera, respecto a la robustez del controlador de la parte 1, se implementaron varias estrategias para tener un controlador robusto. Primeramente se limitaron los comandos de control mediante saturaciones de velocidad lineal y angular evitando que el robot recibiera órdenes demasiado grandes que provocan oscilaciones. Consecuentemente, se estableció una tolerancia angular que obliga al robot a corregir primero su orientación antes de avanzar lo cual disminuye

errores al momento de hacer la trayectoria. Asimismo, se utilizó una tolerancia de llegada ya que la odometría puede generar desviaciones respecto a la posición real del robot y finalmente se normalizó el ángulo para que esté siempre permaneciera en un cierto rango definido.

Parte 2

Para la segunda parte del reto se implementó un sistema de generación y seguimiento de trayectorias mediante los nodos *path_generator_node* y *controller_path* debido a que el objetivo de este segundo ejercicio era que el robot pudiera recibir metas de forma secuencial publicadas por otro nodo utilizando un mensaje personalizado y que los parámetros de la generación de trayectoria pudiera ser modificada por el usuario. Para ello se utilizó nuevamente el mensaje personalizado *MakbetsPose* el cual obtiene las posiciones objetivo en x y y, la velocidad lineal y angular y el modo en el que desea ejecutar el programa el usuario.

Por ende, primeramente se implementó el nodo *path_generator_node* con el fin de crear y publicar rutas en el tópico */goal.* Su funcionamiento se basa en mantener una lista de puntos objetivo y verificar usando la pose recibida desde el tópico */odom* si el robot ya alcanzó la meta o el punto meta. Si esto ocurre, el nodo incrementa el índice de la trayectoria y publica la siguiente meta. De igual manera se implementaron distintos modos de trayectoria configurables mediante parámetros y que el usuario puede cambiar usando rqt reconfigure. A través de rqt_reconfigure el usuario puede: ingresar puntos manuales para hacer cualquier trayectoria; seleccionar figuras preconfiguradas como el triángulo, el rombo y el pentágono; y seleccionar el modo de ejecución del código (siendo modo 1= puntos manuales, modo 2= triangulo, modo 3= rombo, modo 4 = pentágono). También se realizó una verificación de viabilidad de los puntos basada en la distancia entre puntos consecutivos, es decir, cuando un tramo supera los 3m, se consideraba que la trayectoria podía estar fuera de los límites de la pista, por lo que se detenía la ejecución y salía una alerta en la terminal indicando que el punto era o no alcanzable dependiendo de la situación.

Posteriormente se implementó el controlador usando un nodo llamado *controller_path* cuyo funcionamiento conserva la misma lógica del controlador proporcional implementado para la parte 1, pero ahora recibiendo la meta desde el tópico */goal* mediante el mensaje personalizado *MakbetsPose.* Este nodo calcula nuevamente el error de la distancia y el error angular a partir de la pose actual y la meta publicada para luego generar los comandos twist para el tópico */cmd_vel*. En este caso los parámetros podían ser

cambiados por el usuario pero los parámetros base y que tenían mejores resultados fueron los siguientes: *kp_dist = 0.9, kp_theta = 0.3, theta_tolerance = 0.15 y goal_tolerance = 0.1.*

Tomando en cuenta lo anteriormente mencionado, para la ejecución del sistema se implementaron dos archivos launch. El primero llamado *square_controller.launch* sirve para la parte 1 del reto y lanza los nodos *puzzlebot_odometry* y *controller_sqr*. El segundo archivo launch llamado *path_controller.launch* es utilizado para la parte 2 del mini reto e inicia los nodos *puzzlebot_odometry*, *controller_path* y *path_generator_node*. De igual manera en ambos casos se cargó el archivo *puzzlebot_params.yaml* donde se centralizaron las ganancias del controlador, los límites de velocidad, las tolerancias y las metas. Esto permitió organizar de buena manera el reto y modificar el comportamiento del sistema de manera sencilla durante el proceso de pruebas.

Finalmente, para la validación de la solución se incorporó la visualización en tiempo real de la trayectoria que estaba haciendo el robot físicamente mediante matplotlib tanto en el controlador del cuadrado como en el de la trayectoria. Esto permitió observar de manera gráfica la evolución de la posición del robot en tiempo real y compararla con la forma esperada.

Resultados

Para evaluar que todo funciona correctamente, se realizaron diferentes pruebas, tanto para el primer modo, es decir, el cuadrado, y para el segundo modo, donde el usuario ingresa un número, correspondiente a 4 modos diferentes, el primero siendo una trayectoria con 4 coordenadas, y los otros tres modos son figuras geométricas predeterminadas.

En el modo 2, la figura que tiene por default es un cuadrado, por lo que el enfoque fue corregir el modo dos antes que el modo 1. El funcionamiento puede observarse en el siguiente video, donde se puede observar la gráfica de la trayectoria en tiempo real, al igual que la terminal, donde se imprimen la posición del robot, las velocidades lineales y angulares del robot, y el error, tanto de la distancia como del ángulo, todo esto en tiempo real. Igualmente se observa el comportamiento del robot físico, el cual realiza las trayectorias de la manera esperada, con una histéresis definida igualmente en el código del robot, esto principalmente para evitar comportamientos no deseados una vez que el error va disminuyendo. Esto es importante, pues el robot sí respeta ese rango que se implementó, específicamente se tratan de ±10 cm para la distancia.

Video de resultados: https://youtu.be/cdsBn3Jpiwc

Esto igualmente puede observarse en las imágenes 1 y 2, las cuales sirvieron de referencia para el robot físico y como apoyo visual en la estimación de la posición del robot. Observar la trayectoria permitió darnos una idea de que cambiar en los parámetros, específicamente en el controlador P, que se implementó tanto para la velocidad como para el ángulo.

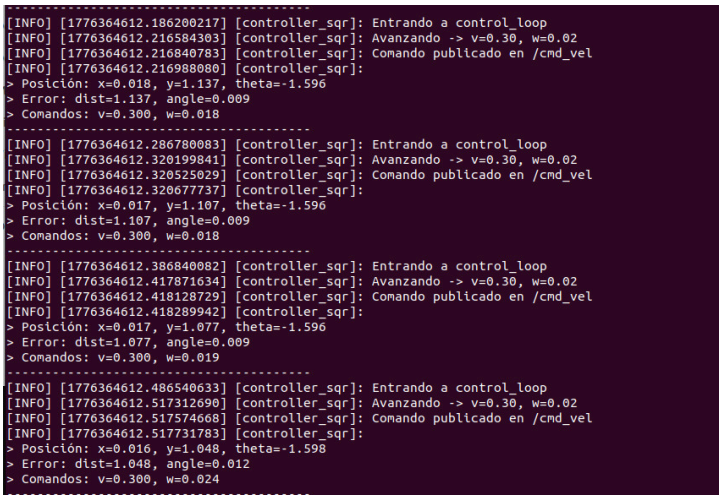

Imagen 1. Terminal donde se observa las velocidades angulares y lineales, el error y la posición del robot.

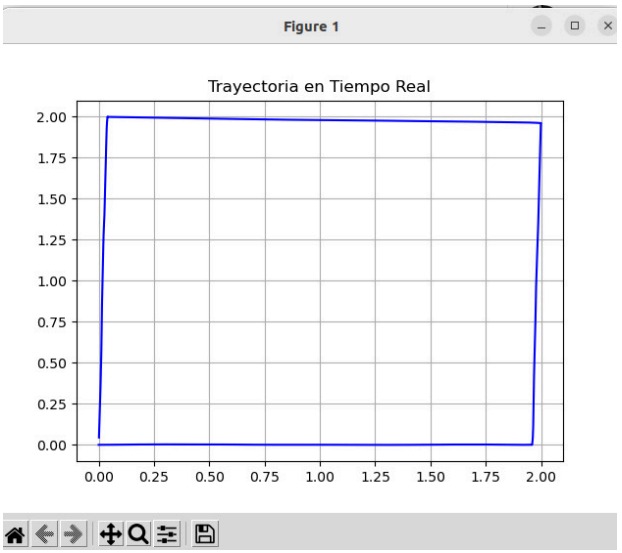

Imagen 2. Trayectoria del cuadrado realizada en tiempo real.

Mecanismos de robustez

Al tratarse de un sistema de control en lazo cerrado, es decir con retroalimentación de los encoders en los motores, igualmente fue necesario implementar mecanismos que aumentaran la robustez del sistema.

- Primeramente fue necesario limitar las velocidades, pues al superar cierta velocidad, el robot puede sufrir un reinicio, perdiendo la conexión con la computadora y

realizando movimientos no deseados. Específicamente, los valores de las velocidades deben estar dentro del rango (0.005, 0.3] m/s para la velocidad lineal, y dentro del rango (0.005, 0.72] rad/s para la velocidad angular.

- Igualmente fue necesario normalizar el ángulo, ya que sin esto el robot puede hacer giros necesarios de más de 180°, permitiendo que los giros que hace sean los mínimos necesarios para las trayectorias tanto predefinidas, como las dadas por el usuario.
- Finalmente, fue sumamente necesario implementar pausas entre los movimientos, es decir, pausas antes de avanzar en línea recta y pausas antes de los giros, esto principalmente para evitar el efecto de transiciones abruptas y mejorando la precisión del siguiente movimiento.

Conclusiones

El desarrollo de este reto permitió validar el sistema de control en lazo cerrado, específicamente para la navegación autónoma del robot, demostrando entonces la importancia de la retroalimentación proveniente de los encoders de los motores, evidenciando que la odometría es fundamental a la hora de corregir errores en tiempo real, mejorando la precisión en el seguimiento de trayectorias.

El comportamiento estable y libre de ruido se dio gracias a la implementación del controlador proporcional, saturación de las velocidades, normalización del ángulo, e igualmente el uso de tolerancias en el error de la distancia y del ángulo, además de tener en mente también las limitaciones físicas del robot y del sistema.

En adición a esto, se comprobó la importancia de la sintonización de los parámetros del controlador, pues afectan en el comportamiento y desempeño del sistema, ya que impactan en la rapidez de la respuesta y sobre todo en la reducción del error. Las pruebas que se realizaron dieron pie a un mejor análisis del comportamiento del sistema y en la identificación de las áreas de mejora.

Finalmente, esta actividad permite sentar las bases necesarias para futuras implementaciones de controladores más complejos, es decir PI o PID, mejoras en el cálculo de la localización y el uso de los sensores en el lazo cerrado, reafirmando la importancia del controlador en lazo cerrado en sistemas autónomos.

Referencias

1. Ramirez, K. (s.f). Odometría. https://www.kramirez.net/Robotica/Material/Presentaciones/Odometria.pdf
2. Manchester Robotics (2026). Autonomous Systems. https://github.com/ManchesterRoboticsLtd/TE3002B_Intelligent_Robotics_Implementation_2026/blob/main/Week3/Presentations/PDF/MCR2_Closed_Loop_Control_v3.pdf
3. Maloy, G. (2024). ¿Qué es un controlador PID? https://dewesoft.com/es/blog/que-es-un-controlador-pid